

Reporte de Práctica: Sistema CRUD

Pokémon con Java y MySQL

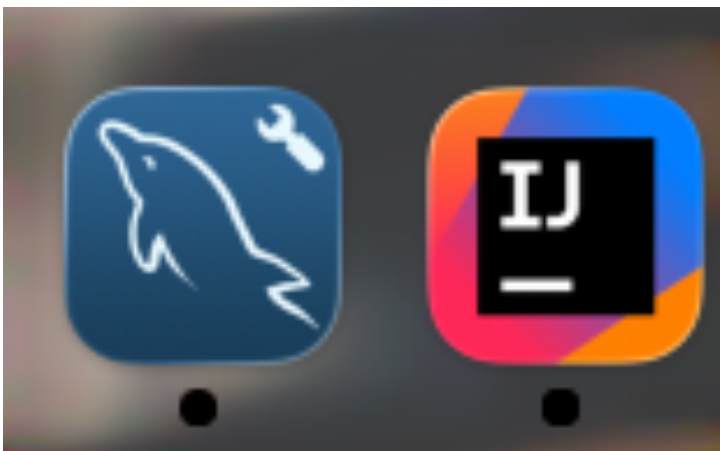
Alumno: Adrian Valenzuela

Institución: Universidad Tecnológica de Torreón (UTT)

Carrera: Desarrollo de Software Multiplataforma

1. Introducción

El presente documento detalla el proceso de creación de un sistema de gestión (CRUD) para una Pokedex y una Mochila, integrando una base de datos relacional en MySQL con una aplicación de consola desarrollada en Java mediante el IDE IntelliJ IDEA.



2. Configuración de la Base de Datos (MySQL Workbench)

Paso 1: Se procedió a la creación del esquema principal denominado pokemon y las tablas correspondientes (mochila) para almacenar la información de las criaturas y los objetos de la mochila.

The screenshot displays the MySQL Workbench interface. The 'SCHEMAS' sidebar on the left shows the 'pokemon' schema expanded, with the 'mochila' table selected. The main workspace shows the 'mochila' table structure with the following columns:

Column	Datatype	PK	NN	UQ	B...	UN	ZF	AI	G	Default / Expression
idmochila	INT	☒	☒	☐	☐	☐	☐	☒	☐	
curas	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
poke balls	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
piedras evolu...	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
mochilacol	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
<click to edit>		☐	☐	☐	☐	☐	☐	☐	☐	

Below the table structure, the 'Column details' panel shows the configuration for the 'mochilacol' column:

- Column Name: mochilacol
- D: VARCHAR(45)
- Charset/Collation: Default Charset
- Def: ☐ S ☐ T ☐ O ☐ R
- ☐ VIRTUAL
- ☐ P ☐ r ☐ i ☐ n ☐ a ☐ r ☐ y ☒ K ☐ e ☐ y
- ☒ Not NULL
- ☐ Unique

The 'Action Output' panel at the bottom shows the results of the table creation:

	Time	Response	Duration / Fetch Time
1	12:14:31	C 1 row(s) affected	0.020 sec
2	08:37:20	A Changes applied	
3	08:39:50	A Changes applied	
4	09:24:02	A Changes applied	

At the bottom of the window, a status bar indicates 'Active schema changed to pokemon'.

3. Conexión Java-MySQL

Paso 2: Se configuró la clase de conexión en Java utilizando el driver JDBC para establecer el puente con el servidor local de MySQL.

```
1 package gim.Conexion;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class Conexion { 4 usages
8     public static Connection getConnection() { 2 usages
9         Connection cox = null;
10        var baseDatos = "pokemon";
11        var url = "jdbc:mysql://localhost:3306/" + baseDatos;
12        try {
13            cox = DriverManager.getConnection(url, user: "root", password: "adrianvalenzuela01");
14            System.out.println("¡Conexión exitosa!");
15        } catch (SQLException e) {
16            System.out.println("Error de conexión: " + e.getMessage());
17        }
18        return cox;
19    }
20 }
21
```

4. Creación de Modelos (Clases POJO)

Paso 3: Se crearon clases para representar los objetos del sistema, permitiendo manipular los datos de forma orientada a objetos antes de enviarlos a la base de datos.

Pokemon.java:

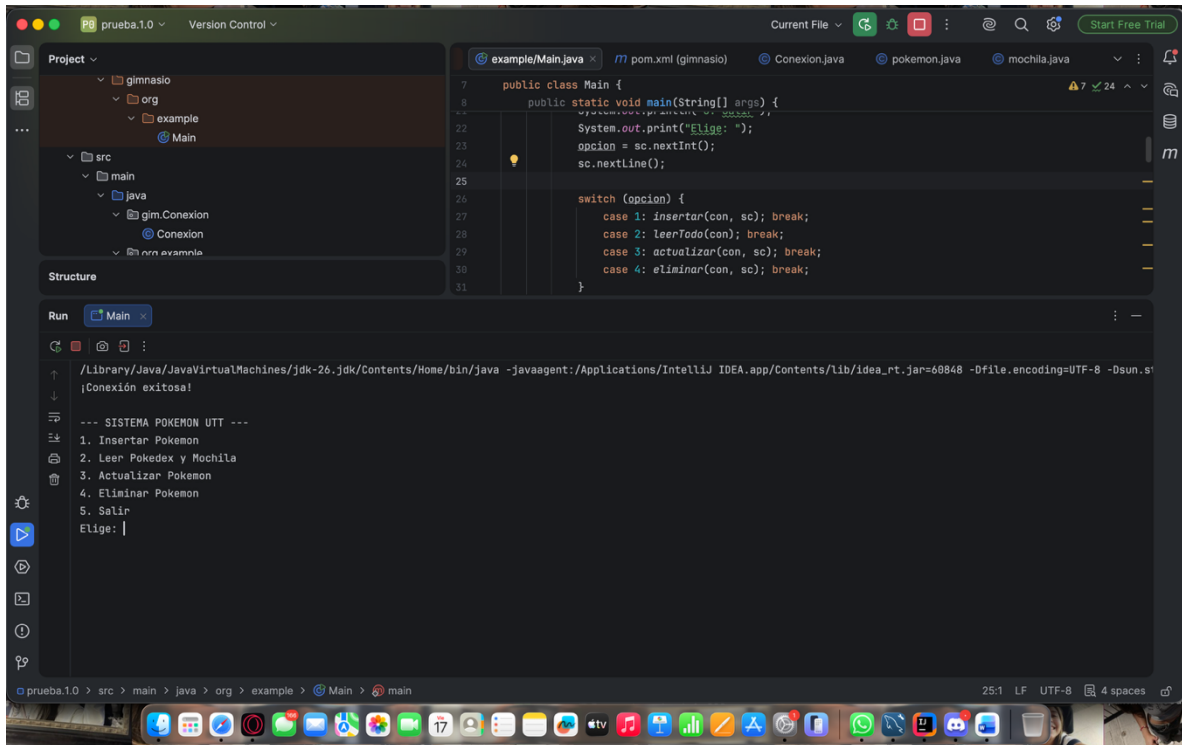
```
1 package org.example;
2
3 public class pokemon { no usages
4     public int id; no usages
5     public String nombre; 1 usage
6     public String elemento; 1 usage
7     public int vida; 1 usage
8     public int daño; 1 usage
9
10    public pokemon(String nombre, String elemento, int vida, int daño) { no usages
11        this.nombre = nombre;
12        this.elemento = elemento;
13        this.vida = vida;
14        this.daño = daño;|
15    }
16 }
```

Mochila.java

```
1 package org.example;
2
3 public class mochila { no usages
4     public int id; no usages
5     public String curas; 1 usage
6     public String pokeBalls; 1 usage
7     public String piedras; 1 usage
8
9     public mochila(String curas, String pokeBalls, String piedras) { no usages
10        this.curas = curas;
11        this.pokeBalls = pokeBalls;
12        this.piedras = piedras;
13    }
14 }
```

5. Implementación del CRUD (Main.java)

Paso 4: Se programó el menú principal que permite al usuario realizar las cuatro operaciones fundamentales: Insertar, Leer, Actualizar y Eliminar registros.



Lógica de Inserción:

```
String sql = "INSERT INTO pokemones (name, element, vida, daño) VALUES
```

```
(?, ?, ?, ?)";
```

```
try (PreparedStatement ps = con.prepareStatement(sql)) {
```

```
ps.setString(1, nombre);
```

```
ps.setString(2, elemento);
```

```
ps.setInt(3, vida);
```

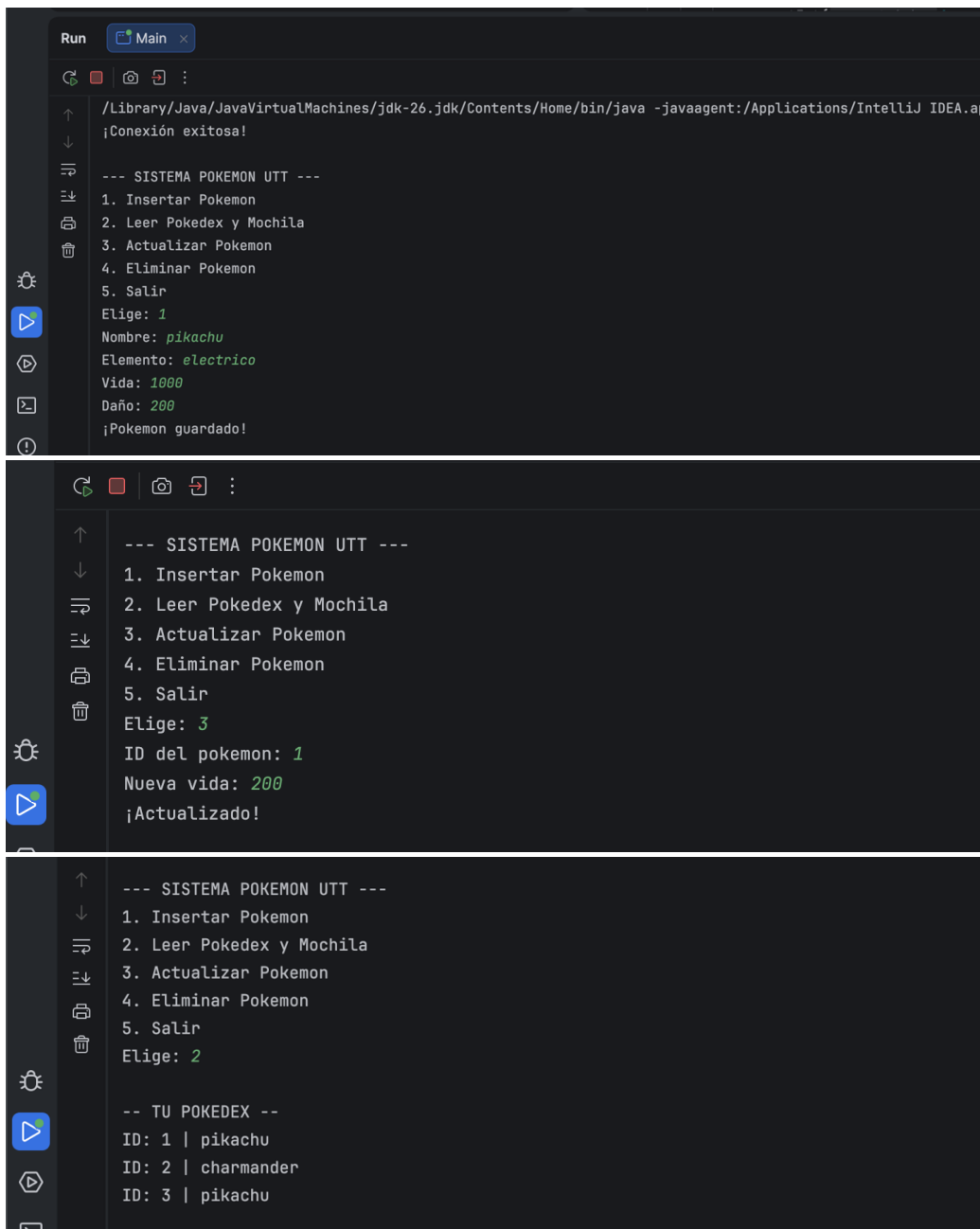
```
ps.setInt(4, daño);
```

```
ps.executeUpdate();
```

```
}
```

7. Conclusión

La práctica permitió comprender el flujo de datos entre una aplicación de escritorio y un servidor de bases de datos. Se logró implementar exitosamente la arquitectura CRUD, asegurando que la información no se pierda al cerrar el programa.



```
Run Main x
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA.a
¡Conexión exitosa!

--- SISTEMA POKEMON UTT ---
1. Insertar Pokemon
2. Leer Pokedex y Mochila
3. Actualizar Pokemon
4. Eliminar Pokemon
5. Salir
Elige: 1
Nombre: pikachu
Elemento: electrico
Vida: 1000
Daño: 200
¡Pokemon guardado!

--- SISTEMA POKEMON UTT ---
1. Insertar Pokemon
2. Leer Pokedex y Mochila
3. Actualizar Pokemon
4. Eliminar Pokemon
5. Salir
Elige: 3
ID del pokemon: 1
Nueva vida: 200
¡Actualizado!

--- SISTEMA POKEMON UTT ---
1. Insertar Pokemon
2. Leer Pokedex y Mochila
3. Actualizar Pokemon
4. Eliminar Pokemon
5. Salir
Elige: 2

-- TU POKEDEX --
ID: 1 | pikachu
ID: 2 | charmander
ID: 3 | pikachu
```